Unit 4:
Object-Oriented Programming

Department of Artificial Intelligence & Machine Learning (AIML)

❖ Introduction to Object-Oriented Programming (OOP)

Object-Oriented Programming is a programming paradigm that organizes software using objects instead of functions and logic. An object represents a real-world entity that contains both data (attributes) and behaviour (methods).

❖ Definition

Object-Oriented Programming (OOP) is a programming approach in which programs are designed using objects that contain both data and methods to perform operations on that data.

❖ Why Do We Need OOP?

Earlier programming languages mainly followed **Procedural Programming**, where the focus was on writing functions. As software projects became larger and more complex, procedural programming created many problems.

Problems in Procedural Programming

- Difficult to maintain large programs
- Code duplication
- Low code reusability
- Data is not secure
- Difficult to debug
- Difficult to modify existing code

OOP Solves These Problems By

- Organizing code into classes and objects
- Promoting code reusability
- Improving security through data hiding
- Making programs modular
- Simplifying maintenance
- Supporting real-world modelling

Example: Imagine a **Car**.

A car has:

Attributes (Data)

- Brand
- Color
- Speed
- Fuel Type

Behaviour (Methods)

- Start()
- Stop()
- Accelerate()
- Brake()

Similarly, in Java:

```
class Car {
    String brand;
    String  color;
    void start() {
        System.out.println("Car Started");
    }
    void stop() {
        System.out.println("Car Stopped");
    }
}
```

Here,

- Car → Class
- Brand, Color → Attributes
- Start(), Stop() → Methods

❖ **What is a Class?**

A **Class** is a blueprint or template used to create objects.

It defines:

- Variables (Attributes)
- Methods (Functions)

Example:

```
class Student {
    String name;
    int age;
    void study() {
        System.out.println("Student is studying");
    }
}
```

❖ **What is an Object?**

An object is an instance of a class.

When memory is allocated for a class, it becomes an object.

Example

```
Student s1 = new Student();
```

Here,

- Student → Class
- s1 → Object

❖ Constructors in Java

Q. What is a Constructor?

A constructor is a special method that is automatically called when an object is created. It is used to initialize the object.

Key Points

- Constructor name must be the same as the class name.
- It has no return type (not even void).
- It is called automatically when an object is created.
- It can be overloaded.

Syntax:

```
class Student {
        Student() {
                System.out.println("Constructor Called");
        }
}
```

Example:

```
class Student {
        Student() {
                System.out.println("Object Created");
         }
        public static void main(String[] args) {
                 Student s1 = new Student();
        }
}
```

Types of Constructors

1. Default Constructor

- Provided automatically by Java if no constructor is written.

2. No-Argument Constructor

- Created by the programmer without parameters.

```
Student() {
        System.out.println("Hello");
}
```

3. Parameterized Constructor

- Accepts parameters to initialize object values.

```
class Student {
        String name;
        Student(String n) {
                name = n;
         }
}
```

❖ Access Specifiers (Access Modifiers)

Q. What are Access Specifiers?

Access specifiers control **who can access** a class, variable, method, or constructor.

Types of Access Specifiers

1. Public (public)

- Accessible from anywhere.

```
public int age = 20;
```

2. Private (private)

- Accessible only within the same class.

```
private int age = 20;
```

3. Protected (protected)

- Accessible within the same package and by subclasses.

```
protected void display() {
}
```

4. Default (No Keyword)

- Accessible only within the same package.

```
String name = "Rahul";
```

❖ Four Pillars of Object-Oriented Programming

Object-Oriented Programming (OOP) is based on four main pillars:

1. Encapsulation
2. Abstraction
3. Inheritance
4. Polymorphism

1. Encapsulation

Encapsulation is one of the four pillars of Object-Oriented Programming (OOP). It is the process of combining data (variables) and methods (functions) into a single unit called a class and protecting the data from unauthorized access by using access modifiers.

Encapsulation is the process of wrapping data and methods into a single class while protecting the data using access modifiers.

❖ Why Do We Need Encapsulation?

Encapsulation is needed to:

- Protect data from unauthorized access.
- Prevent accidental modification of data.
- Improve data security.
- Allow controlled access to variables.
- Make the program easy to maintain and modify.

❖ How to Achieve Encapsulation?

Encapsulation is achieved by following three simple steps:

1. Declare the instance variables as private.
2. Create public getter methods to read the data.
3. Create public setter methods to update the data.

Example:

```
class Student {
        private String name;
        public void setName(String name) {
                this.name = name;
        }
        public String getName() {
                return name;
        }
}
public class Main {
        public static void main(String[] args) {
                Student s = new Student();
                s.setName("Rahul");
                System.out.println(s.getName());
        }
}
```

Output: Rahul

Example: Bank Account

A customer cannot directly change the account balance.
Instead, they use methods such as:

- deposit()
- withdraw()
- checkBalance()

The actual balance is kept private, ensuring that it cannot be modified without proper validation.

❖ **Advantages of Encapsulation**

- Provides data security.
- Prevents unauthorized access to data.
- Supports data hiding.
- Improves code maintainability.
- Allows controlled access to variables.
- Makes programs more flexible and reusable.

❖ **Disadvantages of Encapsulation**

- Requires writing additional getter and setter methods.
- Slightly increases the amount of code.
- Can introduce a small performance overhead due to method calls.

❖ Abstraction

1. What is Abstraction?

Ans. Abstraction is one of the four pillars of Object-Oriented Programming (OOP). It is the process of hiding implementation details and showing only the essential features to the user.

In simple words, the user knows what an object does but does not know how it works internally.

Example: ATM Machine, Car, Mobile Phone.

❖ Why Do We Need Abstraction?

- Hides unnecessary implementation details.
- Reduces program complexity.
- Improves security.
- Makes code easier to maintain.
- Allows developers to focus only on important functionalities.

❖ Ways to Achieve Abstraction

Java provides two ways to achieve abstraction:

1. Abstract Class
2. Interface

1. Abstract Class

An abstract class is a class that is declared using the abstract keyword. It cannot be instantiated (you cannot create its object directly).
It is mainly used to provide a common base class for other classes.

Syntax

```
abstract class Vehicle {

}
```

Features of Abstract Class

- Declared using the abstract keyword.
- Cannot create an object directly.
- Can contain abstract methods and concrete methods.
- Can have constructors.
- Can have instance variables.
- Can have static methods.
- Can have final methods.
- A child class must implement all abstract methods unless it is also abstract.

❖ Abstract Method

An abstract method is a method that has no body (implementation).
It ends with a semicolon (;).

Syntax

```
abstract void start();
```

Example:

```
abstract class Vehicle {
        abstract void start();
        void stop() {
                System.out.println("Vehicle Stopped");
        }
}
class Car extends Vehicle {
        void start() {
                System.out.println("Car Started");
         }
}
public class Main {
        public static void main(String[] args) {
                Car c = new Car();
                c.start();
                c.stop();
        }
}
```

Output

```
Car Started
Vehicle Stopped
```

Advantages of Abstract Class

- Supports code reuse.
- Can provide common implementation.
- Reduces duplicate code.
- Easy to maintain.

❖ Interface

An Interface is a blueprint of a class that defines a set of methods that implementing classes must provide.
It is declared using the interface keyword.
A class uses the implements keyword to implement an interface.

Syntax

```
interface Vehicle {

}
```

❖ Features of Interface

- Declared using the interface keyword.
- Cannot create an object directly.
- A class implements an interface using implements.
- Supports multiple inheritance.
- Used to define a common contract.
- Variables are public static final by default.
- Methods are public abstract by default (excluding newer Java features like default and static methods).

Example:

```
interface Vehicle {
        void start();
}
class Car implements Vehicle {
        public void start() {
                System.out.println("Car Started");
        }
}
public class Main {
        public static void main(String[] args) {
        Car c = new Car();
        c.start();
        }
}
```

Output: Car Started

- **Advantages of Interface**
  - Achieves abstraction.
  - Supports multiple inheritance.
  - Promotes loose coupling.
  - Improves code flexibility.
  - Makes applications easier to extend.

- **Inheritance in Java**

Q. What is Inheritance?

Ans. Inheritance is one of the four pillars of Object-Oriented Programming (OOP). It is the process by which one class acquires the properties (variables) and behaviour (methods) of another class.

The class that inherits is called the Child Class (Subclass), and the class whose properties are inherited is called the Parent Class (Superclass).

Inheritance is the process of creating a new class from an existing class so that the new class can reuse the properties and methods of the existing class.

Why Do We Need Inheritance?

Inheritance is used to:

- Reuse existing code.
- Reduce code duplication.
- Improve code maintainability.
- Extend the functionality of an existing class.
- Create a parent-child relationship between classes.

❖ **How to Achieve Inheritance?**

Inheritance is achieved using the **extends** keyword.

Syntax

```
class Parent {
        // Variables and Methods
}
class Child extends Parent {
        // Additional Variables and Methods
}
```

❖ **Example of Inheritance**

```
class Animal {
        void eat() {
                System.out.println("Animal is Eating");
         }
}
class Dog extends Animal {
         void bark() {
                System.out.println("Dog is Barking");
        }
}

public class Main {
        public static void main(String[] args) {
                Dog d = new Dog();
                d.eat();  // Inherited Method
                d.bark();  // Child Method
         }
}
```

Output: Animal is Eating
Dog is Barking

❖ **Types of Inheritance**

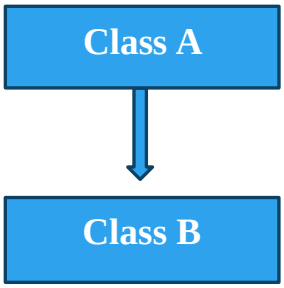
Java conceptually has five types of inheritance.

1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance
4. Multiple Inheritance
5. Hybrid Inheritance

1. Single Inheritance

One child class inherits from one parent class.

Diagram



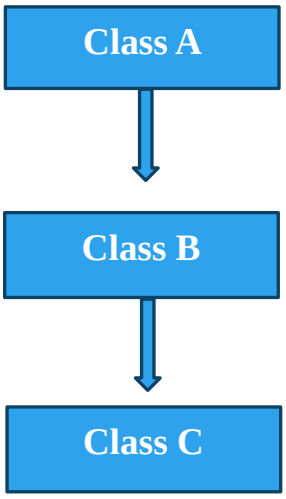
Example:

```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}
class Dog extends Animal {

    void bark() {
        System.out.println("Barking");
    }
}
```

2. Multilevel Inheritance

Ans. A class inherits from another class, which itself inherits from another class.

Diagram



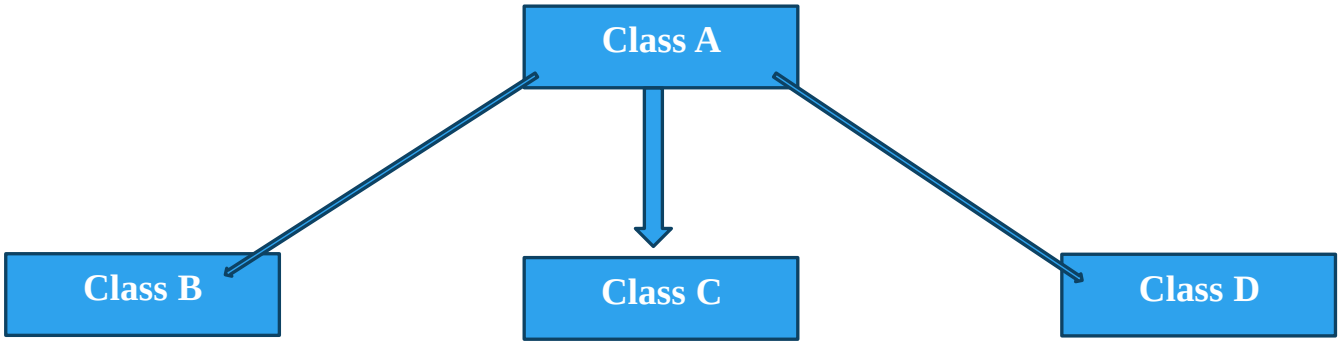
Example:

```
class Animal {
        void eat() {
                System.out.println("Eating");
        }
}
        class Mammal extends Animal {

        void walk() {
                System.out.println("Walking");
        }
}

class Dog extends Mammal {
        void bark() {
                System.out.println("Barking");
        }
}
```

3. Hierarchical Inheritance

Multiple child classes inherit from the same parent class.

Diagram



Example

```
class Animal {
        void eat() {
                System.out.println("Eating");
                }
}
class Dog extends Animal { }
class Cat extends Animal { }
class Cow extends Animal { }
```

4. Multiple Inheritance

A class inherits from more than one parent class.

Does Java Support It?

No. Java does not support multiple inheritance through classes because it can create ambiguity (the Diamond Problem). However, Java supports multiple inheritance using interfaces.

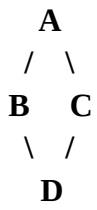
Example

```
interface A {
        void displayA();
}
interface B {
        void displayB();
}
class C implements A, B {
        public void displayA() {
                System.out.println("Interface  A");
        }
        public void displayB() {
                System.out.println("Interface B");
         }
}
```

5. Hybrid Inheritance

Hybrid inheritance is a combination of two or more types of inheritance.

Diagram



Does Java Support It?

Java does not support hybrid inheritance through classes, but it can be achieved using interfaces**.**

❖ The **extends** Keyword

The extends keyword is used to inherit one class from another.

Example

```
class Parent {

}
class Child extends Parent {

}
```

❖ The **super** Keyword

The super keyword refers to the immediate parent class.

It is used to:

- Access parent class variables.
- Call parent class methods.
- Call the parent class constructor.

Example

```
class Animal {
        void sound() {
                System.out.println("Animal Sound");
        }
}
class Dog extends Animal {
        void sound() {
                super.sound();
                System.out.println("Dog Barks");
        }
}
```

❖ The final **Keyword** in Inheritance

The final keyword prevents inheritance. A final class cannot be extended.

Example

```
final class Animal {

}
// Error
class Dog extends Animal {

}
```

❖ **Advantages of Inheritance**

- Promotes code reusability.
- Reduces code duplication.
- Improves maintainability.
- Makes programs easier to extend.
- Supports hierarchical classification.

❖ **Disadvantages of Inheritance**

- Creates tight coupling between classes.
- Changes in the parent class may affect child classes.
- Overuse can make the program more complex.

❖ **Polymorphism in Java**

1. What is Polymorphism?

Polymorphism is one of the four pillars of Object-Oriented Programming (OOP). The word Polymorphism comes from two Greek words:

- Poly = Many
- Morph = Forms

Thus, Polymorphism means "One Interface, Many Forms."
It allows the same method or object to perform different tasks depending on the situation.

❖ **Why Do We Need Polymorphism?**

Polymorphism is used to:

- Increase code flexibility.
- Improve code reusability.
- Reduce code duplication.
- Make programs easier to maintain.
- Allow one interface to represent different behaviors.

❖ **Types of Polymorphism**

Java supports two types of polymorphism:

1. Compile-Time Polymorphism (Static Polymorphism)
2. Run-Time Polymorphism (Dynamic Polymorphism)

4. Compile-Time Polymorphism (Method Overloading)

Compile-Time Polymorphism is achieved using Method Overloading. It is called compile-time because the method to be executed is determined by the compiler during compilation.

It is also known as:

- Static Binding
- Early Binding

Method Overloading

Method Overloading means creating multiple methods with the same name but different parameter lists in the same class.

Rules of Method Overloading

- Method name must be the same.
- Parameters must be different (number, type, or order).
- Return type alone cannot differentiate overloaded methods.

Example

```
class Calculator {
        int add(int a, int b) {
        return a + b;
}
int add(int a, int b, int c) {
        return a + b + c;
}
 double add(double a, double b) {
        return a + b;
}
        }
public class Main {
         public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println(c.add(10, 20));
        System.out.println(c.add(10, 20, 30));
        System.out.println(c.add(10.5, 20.5));
```

```
        }
    }
```
Output : 30
60
31.0

Advantages

- Improves code readability.
- Increases flexibility.
- Reduces the need for multiple method names.

5. Run-Time Polymorphism (Method Overriding)

Run-Time Polymorphism is achieved using Method Overriding. It is called run-time because the method to execute is decided while the program is running.

It is also known as:

- Dynamic Binding
- Late Binding

❖ **Method Overriding**

Method Overriding occurs when a child class provides its own implementation of a method already defined in the parent class.

Rules of Method Overriding

- Parent and child classes are required.
- Method name must be the same.
- Parameters must be the same.
- Return type must be the same (or covariant).
- The access level cannot be more restrictive than in the parent.
- final, private, and static methods cannot be overridden.

Example

```
class Animal {
    void sound() {
        System.out.println("Animal Sound");
    }
}
class Dog extends Animal {
@Override
    void sound() {
        System.out.println("Dog Barks");
    }
}
public class Main {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();
     }
}
```

Output: Dog Barks

Advantages

- Allows dynamic behavior.
- Makes programs flexible.
- Supports code extensibility.
- Improves maintainability.

❖ **Advantages of Polymorphism**

- Increases code reusability.
- Improves flexibility.
- Makes code easier to maintain.
- Supports dynamic method execution.
- Reduces code duplication.

❖ **Disadvantages of Polymorphism**

- Can make code harder to understand for beginners.
- Runtime polymorphism may have a small performance overhead.
- Debugging overridden methods can be more complex.

*****

Important Questions

1. Explain the Access Specifiers in Java with suitable examples.
2. What is a Class? What is an Object? Explain with a suitable Java program.
3. What is a Constructor? Explain the different types of constructors with suitable examples.
4. What is Object-Oriented Programming (OOP)? Explain the four pillars of OOP with suitable examples.
5. What is Encapsulation? Explain Getter and Setter methods with a suitable Java program.
6. What is Inheritance? Explain the different types of inheritance with suitable diagrams and examples.
7. What is the super keyword? Explain its uses with suitable examples.
8. What is Polymorphism? Explain Method Overloading and Method Overriding with suitable examples.
9. Differentiate between Method Overloading and Method Overriding.
10. What is Abstraction? Explain Abstract Class and Interface with suitable examples.
11. Differentiate between an Abstract Class and an Interface.
12. What is a Functional Interface? Explain its features and advantages with a suitable example.
13. What is the this keyword? Explain its uses with suitable examples.
14. What is the Object class in Java? Explain its important methods with suitable examples.
15. What is the final keyword? Explain the use of final variable, final method, and final class with suitable examples.